

COSC 4370 Homework 4

Raphael Tinoco
October 29, 2023

1. Problem Assignment

The task of this assignment was to implement texture mapping using OpenGL.

2. Methods Used

Similar to Homework 3, the **lookAt()** method was used to create the camera transformation, and the **perspective()** method to create the projection matrix.

However, in this homework, an additional buffer was to be created for the texture coordinates. To do this, a few OpenGL methods were used:

- **glGenBuffers(GLsizei n, GLuint *buffers)** - takes in a number of buffer objects and the IDs of the generated buffer objects
- **glBindVertexArray(GLsizei n, GLuint array)** - binds a number of already generated vertex array object(s)
- **glBindBuffer(GLenum target, GLuint buffer)** - binds a buffer object to an intended target of which specifies what kind of data the buffer contains
- **glBufferData(GLenum target, GLsizeiptr size, const GLvoid *data, GLenum usage)** - allocates for the storage of the actual buffer; target = type of data being stored, size relates to the entire size of data, *data is the actual data itself, and usage states how the data will be used
- **glVertexAttribPointer(GLuint index, GLint size, GLenum type, GLboolean normalized, GLsizei stride, const GLvoid *pointer)** - defines an array of vertex attribute data; index specifies the index for a vertex attribute (i.e. position, color, texture, etc.), size indicates the number of components (i.e. x,y,z for position => size = 3), type is data type, normalized specifies whether values are normalized or not, stride indicates byte offset between consecutive attributes in the buffer, and *pointer points to where the attribute data starts in memory
- **glEnableVertexAttribArray(GLuint index)** - enables the specified vertex attribute array
- **texture(sampler2D myTextureSampler, vec2 UV)** - samples color of myTextureSampler according to UV texture coordinates

Also, the texture was requested to be bounded. To do this, we called the **loadDDS(const char *imagepath)** function already given to us within main.cpp. This function loads and creates a texture given a DDS image file, and also returns a texture ID.

3. Implementation Details

Camera and Perspective Transformations

To get started with implementing texture mapping, I had to add in code to create the camera and perspective transformations. These were simple enough, as all I had to do was pass in a few vectors (position, view direction, up direction)) into the `lookat()` function for the camera transformation.

This function would then return the camera transformation matrix. For the perspective transformation, I used the `perspective()` function and passed in the following parameters: `glm::radians(45.0f)`, `800.0f / 600.0f`, `0.1f`, `100.0f`). In order, they are the field of view, the aspect ratio, the minimum distance from the camera and the max distance from it.

UV Buffer Binding

Following this, I had to set up the UV buffer. To do this, I first created a `GLuint` object (UVBO) and called `glGenBuffers(1, &UVBO)` to generate it as a buffer object. Following the binding of the vertex data, I was then able to call `glBindBuffer(GL_ARRAY_BUFFER, UVBO)` to bind UVBO to `GL_ARRAY_BUFFER`, which is useful for vertex attribute data like positions and texture coordinates, in this case. Then I called and `glBufferData(GL_ARRAY_BUFFER, sizeof(uv), uv, GL_STATIC_DRAW)` to allocate the appropriate data for the size of the uv coordinate array, and to set it as a static draw which indicates that the data will not be frequently modified during different renderings.

Next, I setup the vertex attribute pointer to point to the UV coordinates at location 1 by calling `glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE, 2 * sizeof(GLfloat), (GLvoid *)0)`. Then I enable the vertex attribute array (for my uv coordinates) using `glEnableVertexAttribArray(1)`.

Texture Binding

Next, I had to bind my texture. Reading the rest of `main.cpp`, I noticed a built in function `loadDDS()` which does this step for me. So, I simply set the already existing `textureID` variable as: `textureID = loadDDS("texture.dds")`.

Texture.vs

As this vertex shader is given both vertex position and uv coordinates, two things were to be done.

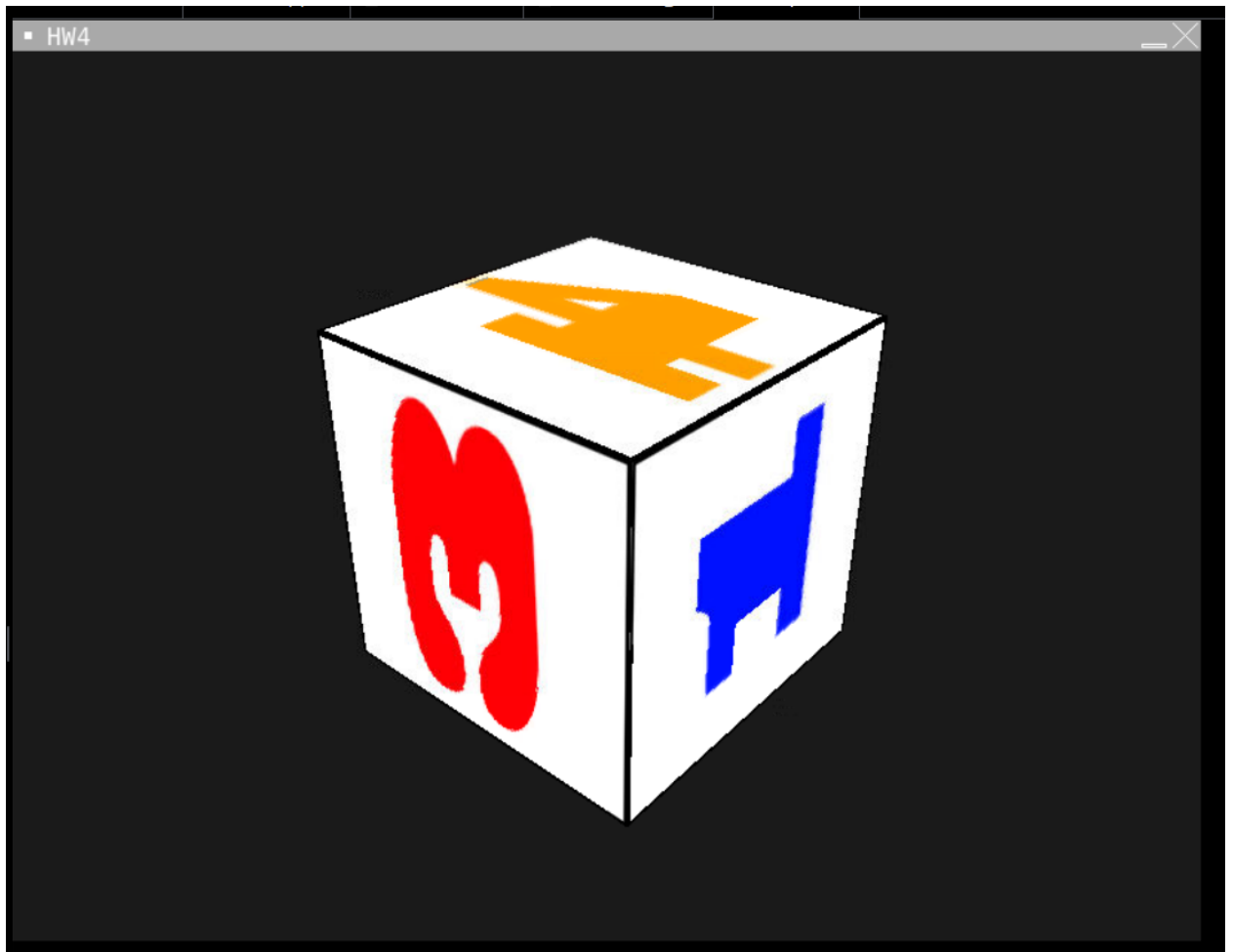
The first was to project the vertex position into clip coordinates by doing the following transformations: `gl_Position = projection * view * model * vec4(position, 1.0f)`.

Next I simply output back the given UV coordinates.

Texture.frag

Finally, this frag file samples the color of the given `myTextureSampler` according to the UV coordinates by using the `texture()` function seen here: `color = texture(myTextureSampler, UV)`. With this, the frag file outputs the color vector.

4. Results



Fully textured cube

5. Resources

- <https://learnopengl.com/Getting-started/Textures>